

Лекция 4

Перегрузка операций и преобразование типов

1 Перегрузка унарных операций

имя_класса operatorΔ (тип_параметра операнд)

```
{  
    // действия  
}
```

имя_класса operator -- ();//Префиксный
имя_класса operator -- (int);//Постфиксный

```
1  #include <iostream>  
2  using namespace std;  
3  
4  class Counter  
5  {  
6      unsigned int count;  
7      public:  
8          Counter() : count(0) { }  
9          Counter(int c) : count(c) { }  
10         unsigned int get_count()  
11         { return count; }  
12  
13         Counter operator++()  
14         { return Counter(++count); }  
15  
16         Counter operator++(int)  
17         { return Counter(count++); }  
18     };  
19
```

```
20  int main()  
21  {  
22      Counter c1, c2;  
23      cout << "c1 = " << c1.get_count() << endl;  
24      cout << "c2 = " << c2.get_count() << endl;  
25      c2 = ++c1;  
26      cout << "c1 = " << c1.get_count() << endl;  
27      cout << "c2 = " << c2.get_count() << endl;  
28      c2 = c1++;  
29      cout << "c1 = " << c1.get_count() << endl;  
30      cout << "c2 = " << c2.get_count() << endl;  
31      return 0;  
32  }
```

```
c1 = 0  
c2 = 0  
c1 = 1  
c2 = 1  
c1 = 2  
c2 = 1
```

2 Перегрузка бинарных операций

```
тип operatorΔ(список аргументов)
{
    // действия
}
```

Перегруженные бинарные операции вызываются методом левого операнда. Объект, стоящий справа от знака операции, должен быть передан в функцию в качестве аргумента

```
1  #include <iostream>
2  using namespace std;
3
4  class AB
5  {
6      int A;
7      int B;
8  public:
9      AB() : A(3), B(3) { }
10     AB(int a, int b) : A(a), B(b) { }
11
12     void Show_AB()
13     { cout << A << ", " << B << endl; }
14
15     // сложение объектов
16     AB operator+(AB obj) const
17     {
18         int a = A + obj.A;
19         int b = B + obj.B;
20         return AB(a,b);
21     }
22 };
```

```
23
24 int main()
25 {
26     AB obj1, obj2(4,7), obj3;
27     cout << "obj1 = "; obj1.Show_AB();
28     cout << "obj2 = "; obj2.Show_AB();
29     obj3 = obj1 + obj2;
30     cout << "obj3 = "; obj3.Show_AB();
31     return 0;
32 }
```

```
obj1 = 3,3
obj2 = 4,7
obj3 = 7,10
```

1 аршин = 16 вершков = 71,12 см

```
1 #include <iostream>
2 using namespace std;
3
4 class Rasst // класс русских мер длины 18 века
5 {
6     int arshin;
7     int vershok;
8 public:
9     Rasst() : arshin(0), vershok(0) { }
10    Rasst(int ar, int ver) : arshin(ar), vershok(ver) { }
11
12    void Get_Ras()
13    {
14        cout << "\nВведите аршины: "; cin >> arshin;
15        cout << "Введите вершки: "; cin >> vershok;
16    }
17
18    void Show_Ras()
19    { cout << arshin << "~" << vershok << endl; }
20
21    Rasst operator+(Rasst) const; // сложение длин
22    bool operator<(Rasst d2) const; // сравнение длин
23    void operator+=(Rasst d2); // сложение с присваиванием
24};
```

```
49 int main()
50 {
51     Rasst ras1, ras3, ras4;
52     ras1.Get_Ras();
53     Rasst ras2(11, 12), ras5(1, 2), ras6(3, 4);
54     ras3 = ras1 + ras2;
55     ras4 = ras1 + ras2 + ras3;
56     ras5 += ras2;
57     ras6 += ras1 + ras2;
58     cout << "ras1 = "; ras1.Show_Ras();
59     cout << "ras2 = "; ras2.Show_Ras();
60     cout << "ras3 = "; ras3.Show_Ras();
61     cout << "ras4 = "; ras4.Show_Ras();
62     cout << "ras5 = "; ras5.Show_Ras();
63     cout << "ras6 = "; ras6.Show_Ras();
64     return 0;
65 }
```

```
25
26 // сложение двух длин
27 Rasst Rasst::operator+(Rasst d2) const
28 {
29     int a = arshin + d2.arshin; // складываем аршины
30     int v = vershok + d2.vershok; // складываем вершки
31     return Rasst(a, v); // создаем и возвращаем объект
32 }
33
34 // сравнение двух длин
35 bool Rasst::operator<(Rasst d2) const
36 {
37     float t1 = arshin + vershok / 16;
38     float t2 = d2.arshin + d2.vershok / 16;
39     return (t1 < t2) ? true : false;
40 }
41
42 // комбинированное сложение
43 void Rasst::operator+=(Rasst d2)
44 {
45     arshin += d2.arshin;
46     vershok += d2.vershok;
47 }
```

```
Введите аршины: 4
Введите вершки: 2
ras1 = 4~2
ras2 = 11~12
ras3 = 15~14
ras4 = 30~28
ras5 = 12~14
ras6 = 18~18
```

3 Понятие дружественной функции

Дружественная функция (friend) – это функция, которая имеет доступ к закрытым членам класса, как если бы она сама была членом этого класса.

```
1  #include <iostream>
2  using namespace std;
3
4  class MyClass
5  {
6      int M;
7  public:
8      MyClass() { M = 77; }
9      // делаем функцию Show_M() дружественной классу
10     friend void Show_M(MyClass &Obj);
11 };
12
13 // Show_M теперь является другом класса MyClass
14 void Show_M(MyClass &Obj)
15 {
16     // и мы имеем доступ к закрытым членам класса
17     cout << Obj.M << endl;
18 }
19
20 int main()
21 {
22     MyClass MC;
23     Show_M(MC);
24     return 0;
25 }
```

77

4 Перегрузка операторов ввода и вывода

```
1  #include <iostream>
2  using namespace std;
3
4  class AB
5  {
6      int A;
7      int B;
8  public:
9      AB() : A(3), B(3) { }
10     AB(int a, int b) : A(a), B(b) { }
11
12     friend ostream& operator << (ostream &out, const AB &Obj);
13     friend istream& operator >> (istream &in, AB &Obj);
14 };
15
16 ostream& operator<< (ostream &out, const AB &Obj)
17 {
18     // Поскольку operator<< является другом класса AB,
19     // то мы имеем прямой доступ к членам AB
20     out << Obj.A << "," << Obj.B;
21     return out;
22 }
23
24 istream& operator >> (istream &in, AB &Obj)
25 {
26     // Поскольку operator>> является другом класса AB,
27     // то мы имеем прямой доступ к членам AB
28     in >> Obj.A;
29     in >> Obj.B;
30     return in;
31 }
```

```
32
33 int main()
34 {
35     AB Obj1(5,7), Obj2;
36     cout << "Obj1 = " << Obj1 << "; Obj2 = " << Obj2 << endl;
37     cin >> Obj1;
38     cout << "Obj1 = " << Obj1 << endl;
39     return 0;
40 }
```

```
Obj1 = 5,7; Obj2 = 3,3
6 34
Obj1 = 6,34
```

5 Преобразование типов

Вид преобразования типа	Процедура в классе назначения	Процедура в исходном классе
Основной → Основной	Встроенные операции преобразования	
Основной → Класс	Конструктор	
Класс → Основной		Операция преобразования
Класс → Класс	Конструктор	Операция преобразования

Преобразование типов от основного к пользовательскому

```
1  #include <iostream>
2  using namespace std;
3
4  class Rasst // класс русских мер длины 18 века
5  {
6      int arshin;
7      int vershok;
8  public:
9      Rasst() : arshin(0), vershok(0) { }
10
11      // конструктор с одним параметром, переводящий метры в аршины и вершки
12      // (конструктор преобразования)
13      Rasst(float meters)
14      {
15          float f_arshin = meters / 0.7112; // переводим в аршины
16          arshin = int(f_arshin); // берем число полных аршинов
17          vershok = 16 * (f_arshin - arshin); // остаток – это вершки
18      }
19
20      friend ostream& operator << (ostream &out, const Rasst &Obj);
21 };
22
23 ostream& operator<< (ostream &out, const Rasst &Obj)
24 {
25     out << Obj.arshin << "~" << Obj.vershok;
26     return out;
27 }
```

ras1 = 4~3
ras2 = 7~0

```
29 int main()
30 {
31     float mtrs = 3;
32     Rasst ras1 = mtrs; // используется
33                        // конструктор,
34                        // переводящий метры
35                        // в аршины и вершки
36     Rasst ras2(5);
37     cout << "ras1 = " << ras1 << endl;
38     cout << "ras2 = " << ras2 << endl;
39     return 0;
40 }
```


Преобразование типов от пользовательского к основному

```
operator тип_в_который_преобразуем()  
{  
    необходимые преобразования;  
    return переменная_нового_типа;  
}
```

```
переменная = static_cast<тип_в_который_преобразуем> (объект);  
переменная = объект;
```

```

1  #include <iostream>
2  using namespace std;
3
4  class Rasst
5  {
6      int arshin;
7      int vershok;
8  public:
9      Rasst(int ar, int ver) : arshin(ar), vershok(ver) { }
10
11      operator float() const // оператор для перевода аршинов в метры
12      {
13          float f_arshin = vershok / 16.0; // вершки в аршины
14          f_arshin = f_arshin + arshin; // прибавляем целые аршины
15          return f_arshin * 0.7112; // переводим в метры
16      }
17 };
18
19 int main()
20 {
21     float mtrs;
22     Rasst ras1(4,3), ras2(7,0), ras3(1,0);
23     mtrs = static_cast<float>(ras1); // используется оператор перевода в метры
24     cout << mtrs << endl;
25     mtrs = ras2; // используется неявный перевод
26     cout << mtrs << endl;
27     mtrs = (float)ras3;
28     cout << mtrs << endl;
29     return 0;
30 }

```

2.97815

4.9784

0.7112

Преобразование типов от пользовательского к пользовательскому: операция преобразования

```
1  #include <iostream>
2  using namespace std;
3
4  class Rasst
5  {
6      int arshin;
7      int vershok;
8  public:
9      Rasst() : arshin(0), vershok(0) { }
10     Rasst(int ar, int ver) : arshin(ar), vershok(ver) { }
11     friend ostream& operator << (ostream &out, const Rasst &Obj);
12 };
13
14 class Metr
15 {
16     float M;
17 public:
18     Metr(float m): M(m) {};
19
20     // операция преобразования типа
21     operator Rasst() const
22     {
23         float f_arshin = M / 0.7112;
24         int a = int(f_arshin);
25         int v = 16 * (f_arshin - a);
26         return Rasst(a, v);
27     }
28 };
```

```
ras1 = 7~0
ras2 = 1~0
```

```
29
30 ostream& operator<< (ostream &out, const Rasst &Obj)
31 {
32     out << Obj.arshin << "~" << Obj.vershok;
33     return out;
34 }
35
36 int main()
37 {
38     Metr met1(5), met2(0.7112);
39     Rasst ras1 = met1;
40     cout << "ras1 = " << ras1 << endl;
41     Rasst ras2;
42     ras2 = met2;
43     cout << "ras2 = " << ras2 << endl;
44     return 0;
45 }
```

Преобразование типов от пользовательского к пользовательскому: конструктор

```
1  #include <iostream>
2  using namespace std;
3
4  class Metr
5  {
6      float M;
7  public:
8      Metr(float m): M(m) {};
9      float Get_M() { return M; }
10 };
11
12 class Rasst
13 {
14     int arshin;
15     int vershok;
16 public:
17     Rasst() : arshin(0), vershok(0) { }
18     Rasst(int ar, int ver) : arshin(ar), vershok(ver) { }
19
20     Rasst(Metr met)
21     {
22         float f_arshin = met.Get_M() / 0.7112;
23         arshin = int(f_arshin);
24         vershok = 16 * (f_arshin - arshin);
25     }
26
27     friend ostream& operator << (ostream &out, const Rasst &Obj);
28 };
29
30 ostream& operator<< (ostream &out, const Rasst &Obj)
31 {
32     out << Obj.arshin << "~" << Obj.vershok;
33     return out;
34 }
35
36 int main()
37 {
38     Metr met1(5), met2(0.7112);
39     Rasst ras1 = met1;
40     cout << "ras1 = " << ras1 << endl;
41     Rasst ras2(met2);
42     cout << "ras2 = " << ras2 << endl;
43     return 0;
44 }
```

ras1 = 7~0
ras2 = 1~0